```
# Required dependencies
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing.numerical_helpers import numeric_dtypes, factor
import seaborn as sns
import matplotlib as plt
%%capture
```

```
# Importing the dataset
df = pd.read_excel('/content/drive/MyDrive/Life Expectancy Data.xlsx')
df.head()
df.tail()
```

	Country Year and Status	Region	Income Group	Life Expectancy	Healthy Life Expectancy	Control	Suicides	Alcohol Consumption	Road Accidents	Obesity	Diabetes	Hypertension	Tobacco/smoke	Sanitation
0	Afghanistan	Asia/Oceania	Low income	54.8	50.4	38.4	2.1	0.1	70.8	5.0	6.4	14.2	108.0	18.0
1	Albania	Europe & Central Asia	Upper middle income	76	68.1	71.4	3.5	6.1	71.3	27.1		17.8	7.5	5.2
2	Algeria	Middle East & North Africa	Lower middle income	77.1	66.2	72.5	2.5	0.03	28.5	27.5	0	18.2	16.5	2.7
3	Andorra	Europe & Central Asia	High income					12.1		23.6		20.7	2.8	
4	Angola	Sub-Saharan Africa	Lower middle income	60.7	52.6	25.6	2.1	5.6	26.7	3.4	20.5	10	12.6	19.1

```
df.shape
```

```
(9806, 14)
```

Data Cleaning

```
df.replace('', np.nan, inplace=True)
```

```
df.replace('', np.nan, inplace=True)
df = df.drop(axis=1)
df
```

	Country Year and Status	Region	Income Group	Life Expectancy	Healthy Life Expectancy	Control	Suicides	Alcohol Consumption	Road Accidents	Obesity	Diabetes	Hypertension	Tobacco/smoke	Sanitation
0	Afghanistan	Asia/Oceania	Low income	54.8	50.4	38.4	2.1	0.1	70.8	5.0	6.4	14.2	108.0	18.0
1	Albania	Europe & Central Asia	Upper middle income	76	68.1	71.4	3.5	6.1	71.3	27.1		17.8	7.5	5.2
2	Algeria	Middle East & North Africa	Lower middle income	77.1	66.2	72.5	2.5	0.03	28.5	27.5	0	18.2	16.5	2.7
3	Andorra	Europe & Central Asia	High income					12.1		23.6		20.7	2.8	
4	Angola	Sub-Saharan Africa	Lower middle income	60.7	52.6	25.6	2.1	5.6	26.7	3.4	20.5	10	12.6	19.1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
9801	Venezuela (Bolivarian Republic of)	Latin America & Caribbean	Upper middle income	73.8	64.4	54.8	2	5	36	23.6	11.5	24.2	17.5	5.2
9802	Viet Nam	East Asia & Pacific	Lower middle income	76.7	66.3	47.4	7.5	0.6	16.6	2.1	3.1	20.7	17.5.4	0.6
9803	Yemen	Middle East & North Africa	Low income	66.6	57.5	47.6	5.8	0.1	26.4	17.1	20.1	24.5	20.5	53.8
9804	Zambia	Sub-Saharan Africa	Lower middle income	54.5	54.4	20.6	7.3	3.6	20.4	5.7	50.7	14.5	50.7.5	28.8
9805	Zimbabwe	Sub-Saharan Africa	Lower middle income	60.7	54.7	46.4	12.1	3.6	21.2	15.6	27.5	14.5	7.6.5	36.4

9806 rows × 14 columns

```
df1 = df[['Life Expectancy','Healthy Life Expectancy','Suicides','Control','Alcohol Consumption','Road Accidents','Obesity',
          'Diabetes','Hypertension','Tobacco/smoke','Sanitation']]
df1 = df1.dropna(how='')
df1
```

	Life Expectancy	Healthy Life Expectancy	Suicides	Control	Alcohol Consumption	Road Accidents	Obesity	Diabetes	Hypertension	Tobacco/smoke	Sanitation
0	54.8	50.4	2.1	38.4	0.1	70.8	5.0	6.4	14.2	108.0	18.0
1	76.0	68.1	3.5	71.4	6.1	71.3	27.1	NaN	17.8	7.5	5.2
2	77.1	66.2	2.5	72.5	0.03	28.5	27.5	0	18.2	16.5	2.7
3	NaN	NaN	NaN	NaN	12.1	NaN	23.6	NaN	20.7	2.8	NaN
4	60.7	52.6	2.1	25.6	5.6	26.7	3.4	20.5	10.0	12.6	19.1
...	...	...	...	...	...	...	...	...	...	...	...
9801	73.8	64.4	2.0	54.8	5.0	36.0	23.6	11.5	24.2	17.5	5.2
9802	76.7	66.3	7.5	47.4	0.6	16.6	2.1	3.1	20.7	17.5.4	0.6
9803	66.6	57.5	5.8	47.6	0.1	26.4	17.1	20.1	24.5	20.5	53.8
9804	54.5	54.4	7.3	20.6	3.6	20.4	5.7	50.7	14.5	50.7.5	28.8
9805	60.7	54.7	12.1	46.4	3.6	21.2	15.6	27.5	14.5	7.6.5	36.4

9806 rows × 11 columns

```
df1[df1['Healthy Life Expectancy']>'Suicides']['Suicides','Control','Alcohol Consumption','Road Accidents','Obesity',
        'Diabetes','Hypertension','Tobacco/smoke','Sanitation']].sample(frac=)
```

	Life Expectancy	Healthy Life Expectancy	Suicides	Control	Alcohol Consumption	Road Accidents	Obesity	Diabetes	Hypertension	Tobacco/smoke	Sanitation
0	54.8	50.4	2.1	38.4	0.1	70.8	5.0	6.4	14.2	108.0	18.0
1	76.0	68.1	3.5	71.4	6.1	71.3	27.1	NaN	17.8	7.5	5.2
2	77.1	66.2	2.5	72.5	0.03	28.5	27.5	0	18.2	16.5	2.7
3	NaN	NaN	NaN	NaN	12.1	NaN	23.6	NaN	20.7	2.8	NaN
4	60.7	52.6	2.1	25.6	5.6	26.7	3.4	20.5	10.0	12.6	19.1
...	...	...	...	...	...	...	...	...	...	...	...
9801	73.8	64.4	2.0	54.8	5.0	36.0	23.6	11.5	24.2	17.5	5.2
9802	76.7	66.3	7.5	47.4	0.6	16.6	2.1	3.1	20.7	17.5.4	0.6
9803	66.6	57.5	5.8	47.6	0.1	26.4	17.1	20.1	24.5	20.5	53.8
9804	54.5	54.4	7.3	20.6	3.6	20.4	5.7	50.7	14.5	50.7.5	28.8
9805	60.7	54.7	12.1	46.4	3.6	21.2	15.6	27.5	14.5	7.6.5	36.4

9806 rows × 11 columns

```
df1.columns
```

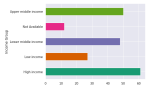
```
Index(['Country Year and Status', 'Region', 'Income Group', 'Life Expectancy',
       'Healthy Life Expectancy', 'Control', 'Suicides', 'Alcohol Consumption',
       'Road Accidents', 'Obesity', 'Diabetes', 'Hypertension',
       'Tobacco/smoke', 'Sanitation'],
      dtype='object')
```

DATA EXPLORATORY THROUGH VISUALIZATIONS

Income Group

```
# plotting Income Group
```

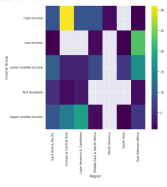
```
from matplotlib import pyplot as plt
df1.groupby('Income Group').size().plot(kind='barh', color=sns.palette(n_colors=1,as_cmap=True,desat=1.0,dark=False,light=False,reverse=False,as_cmap=False))
plt.gca().spines['top', 'right',].set_visible(False)
```



Region vs Income Group

```
# plotting Region vs Income Group
```

```
from matplotlib.mpyplot import plt
%matplotlib inline
import pandas as pd
import numpy as np
df_pivot = df.pivot(index, 0)
df_pivot = df_pivot.fillna(0)
ax = sns.heatmap(df_pivot.T,linewidths=0.1,annot=False,
                 fmt='d',mask,cbar=True,cmap='Pastel1')
ax
ax.set_ylabel('Income Group')
plt.xticks(rotation=90)
plt.xlabel('Region')
_ = plt.yticks(rotation=0)
```




```
columns_to_convert = ['Life Expectancy', 'Healthy Life Expectancy', 'Control', 'Suicides',
                      'Alcohol Consumption', 'Road Accidents', 'Obesity', 'Diabetes',
                      'Hypertension', 'Tobacco/smoke', 'Sanitation']
```

```
# Convert specified columns to float
for col in columns_to_convert:
    df[col] = df[col].astype(float)
```

```
# Verify the updated data types
print(df.dtypes)
```

```
Country Year and Status    object
Region                     object
Income Group               object
Life Expectancy            float64
Healthy Life Expectancy    float64
Control                    float64
Suicides                   float64
Alcohol Consumption        float64
Road Accidents             float64
Obesity                    float64
Diabetes                   float64
Hypertension               float64
Tobacco/smoke              float64
Sanitation                 float64
dtype: object
```

```
df1.columns
```

```
Index(['Country Year and Status', 'Region', 'Income Group', 'Life Expectancy',
       'Healthy Life Expectancy', 'Control', 'Suicides', 'Alcohol Consumption',
       'Road Accidents', 'Obesity', 'Diabetes', 'Hypertension',
       'Tobacco/smoke', 'Sanitation'],
      dtype='object')
```

Aggregating region and Income to see its impact on life expectancy

```
df_ge = df[['Region', 'Income Group', 'Life Expectancy', 'Healthy Life Expectancy']].
df_ge
```

	Region	Income Group	Life Expectancy	Healthy Life Expectancy
0	South Asia	Low income	54.8	50.4
1	Europe & Central Asia	Upper middle income	76.0	68.1
2	Middle East & North Africa	Lower middle income	77.1	66.2
3	Europe & Central Asia	High income	NaN	NaN
4	Sub-Saharan Africa	Lower middle income	60.7	52.6
...	...	...	...	...
9801	Latin America & Caribbean	Upper middle income	73.8	64.4
9802	East Asia & Pacific	Lower middle income	76.7	66.3
9803	Middle East & North Africa	Low income	66.6	57.5
9804	Sub-Saharan Africa	Lower middle income	54.5	54.4
9805	Sub-Saharan Africa	Lower middle income	60.7	54.7

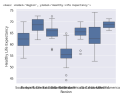
9806 rows × 4 columns

Healthy Life Expectancy by Region

```
# plotting Healthy Life Expectancy by Region
```

```
sns.boxplot(data=df_ge, x='Region', y='Healthy Life Expectancy')
```

```
sns.boxplot(data=df_ge, x='Region', y='Healthy Life Expectancy',
            palette='Pastel1')
```



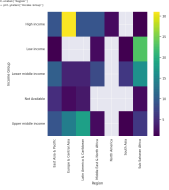

```
df_ge.dtypes
```

```
Region                     object
Income Group               object
Life Expectancy            float64
Healthy Life Expectancy    float64
dtype: object
```

Region vs Income Group

```
# plotting Region vs Income Group
```

```
from matplotlib.mpyplot import plt
import numpy as np
import pandas as pd
df_pivot = df.pivot(index, 0)
df_pivot = df_pivot.fillna(0)
ax = sns.heatmap(df_pivot.T,linewidths=0.1,annot=False,
                 fmt='d',mask,cbar=True,cmap='Pastel1')
ax
ax.set_ylabel('Income Group')
plt.xticks(rotation=90)
plt.xlabel('Region')
_ = plt.yticks(rotation=0)
```




```
from matplotlib.mpyplot import plt
df_ge['Life Expectancy'].hist(bins=20, color='Pastel1', ).set_ylabel('Life Expectancy')
plt.gca().spines['top', 'right',].set_visible(False)
```

```
from matplotlib.mpyplot import plt
df_ge['Life Expectancy'].plot(kind='hist', bins=40, color='Life Expectancy Birth')
plt.gca().spines['top', 'right',].set_visible(False)
```

```
mean_lig = df_ge.groupby('Region')['Life Expectancy', 'Healthy Life Expectancy'].mean()
mean_lig
```

Region	Life Expectancy	Healthy Life Expectancy
East Asia & Pacific	71.000000	64.700000
Latin America & Caribbean	70.277778	62.175556
Middle East & North Africa	70.000000	63.100000
North America	78.000000	67.200000
Sub-Saharan Africa	55.000000	50.600000

```
import numpy as np

data = {'A': [47.000000000000, 48.000000000000,
              '<0', '<0', 50.000000000000,
              '<0', 51.000000000000, 52.000000000000,
              '<0', 53.000000000000, 54.000000000000,
              '<0', 55.000000000000, 56.000000000000],}
```

```
import numpy as np
df1 = df[['Life Expectancy', 'Healthy Life Expectancy']]
healthy_life_expectancy = df1['Healthy Life Expectancy']
```

```
fig, ax = plt.subplots()
ax.hist(healthy_life_expectancy, bins=10, color='Life Expectancy Birth')
ax.set_xlabel('Healthy Life Expectancy', fontsize=14)
ax.set_ylabel('Frequency', fontsize=14)
ax.set_title('Healthy Life Expectancy')
plt.show()
```

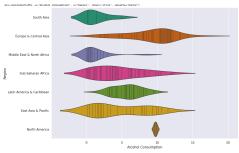
```
# Scatter plot of life expectancy and healthy life expectancy by income group
sns.lmplot(x='Life Expectancy', y='Healthy Life Expectancy', hue='Income Group')
plt.show()
```

```
df6 = df5[['Income Group', 'Region', 'Alcohol Consumption']]
df6
```

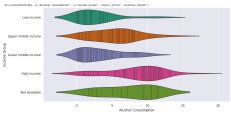
	Income Group	Region	Alcohol Consumption
0	Low-income	South Asia	0.1
1	Upper-middle-income	Europe & Central Asia	9.7
2	Lower-middle-income	Middle East & North Africa	2.6
3	High-income	Europe & Central Asia	11.7
4	Lower-middle-income	Sub-Saharan Africa	6.8
...	...	...	...
186	Upper-middle-income	Latin America & Caribbean	5.8
188	Lower-middle-income	East Asia & Pacific	8.0
189	Low-income	Middle East & North Africa	0.1
190	Lower-middle-income	Sub-Saharan Africa	3.6
191	Lower-middle-income	Sub-Saharan Africa	3.8

188 rows × 3 columns

```
from seaborn.categorical import _CategoricalPlotter, catplot
import seaborn as sns
figsize = (10, 5.5 * len(df6['Region'].unique()))
g = sns.catplot(data=df6, kind="violin",
x="Alcohol Consumption", y="Region", hue=None,
order=['South Asia', 'Europe & Central Asia',
'Middle East & North Africa', 'Sub-Saharan Africa',
'Latin America & Caribbean', 'East Asia & Pacific',
'North America'],
    /opt/hostedtoolcache/Python/3.7.12/x64/lib/pkg
    Passing (positio without assigning \'hue\' is deprecated and will be removed in v0.14.0. Assign the \'y\' variable to \'hue\' and set \'legend=False\' for the same effect.
      sns.violinplot(data=df6, x="Alcohol Consumption", y="Region", order=['South
```




```
from seaborn.categorical import _CategoricalPlotter, catplot
import seaborn as sns
figsize = (10, 5.5 * len(df6['Income Group'].unique()))
g = sns.catplot(data=df6, kind="violin",
x="Alcohol Consumption", y="Income Group", hue=None,
order=['Low-income', 'Upper-middle-income',
'Lower-middle-income', 'High-income', 'Not Available'],
    /opt/hostedtoolcache/Python/3.7.12/x64/lib/pkg
    Passing (positio without assigning \'hue\' is deprecated and will be removed in v0.14.0. Assign the \'y\' variable to \'hue\' and set \'legend=False\' for the same effect.
      sns.violinplot(data=df6, x="Alcohol Consumption", y="Income Group", order=['Low-
```

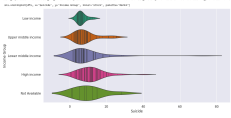



```
df8 = df6[['Region', 'Income Group', 'Suicide']]
df8
```

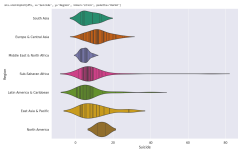
	Region	Income Group	Suicide
0	South Asia	Low-income	4.7
1	Europe & Central Asia	Upper-middle-income	14.5
2	Middle East & North Africa	Lower-middle-income	2.3
3	Europe & Central Asia	High-income	9.05
4	Sub-Saharan Africa	Lower-middle-income	6.7
...	...	...	...
186	Latin America & Caribbean	Upper-middle-income	4.9
188	East Asia & Pacific	Lower-middle-income	7.5
189	Middle East & North Africa	Low-income	5.6
190	Sub-Saharan Africa	Lower-middle-income	7.3
191	Sub-Saharan Africa	Lower-middle-income	5.1

188 rows × 3 columns

```
from seaborn.categorical import _CategoricalPlotter, catplot
import seaborn as sns
figsize = (10, 5.5 * len(df8['Income Group'].unique()))
g = sns.catplot(data=df8, kind="violin",
x="Suicide", y="Income Group", hue=None,
order=['Low-income', 'Upper-middle-income',
'Lower-middle-income', 'High-income', 'Not Available'],
    /opt/hostedtoolcache/Python/3.7.12/x64/lib/pkg
    Passing (positio without assigning \'hue\' is deprecated and will be removed in v0.14.0. Assign the \'y\' variable to \'hue\' and set \'legend=False\' for the same effect.
      sns.violinplot(data=df8, x="Suicide", y="Income Group", order=['Low-income',
```




```
from seaborn.categorical import _CategoricalPlotter, catplot
import seaborn as sns
figsize = (10, 5.5 * len(df8['Region'].unique()))
g = sns.catplot(data=df8, kind="violin",
x="Suicide", y="Region", hue=None,
order=['South Asia', 'Europe & Central Asia',
'Middle East & North Africa', 'Sub-Saharan Africa',
'Latin America & Caribbean', 'East Asia & Pacific',
'North America'],
    /opt/hostedtoolcache/Python/3.7.12/x64/lib/pkg
    Passing (positio without assigning \'hue\' is deprecated and will be removed in v0.14.0. Assign the \'y\' variable to \'hue\' and set \'legend=False\' for the same effect.
      sns.violinplot(data=df8, x="Suicide", y="Region", order=['South
```



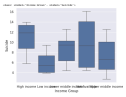

```
mean_ca = df8.groupby('Income Group', 'Region')['Suicide'].mean().reset_index()
mean_ca
```

	Income Group	Region	Suicide
0	High-income	East Asia & Pacific	7.5166667
1	High-income	Europe & Central Asia	9.4666667
2	High-income	Latin America & Caribbean	5.5166667
3	High-income	Middle East & North Africa	5.7166667
4	High-income	North America	7.7666667
5	Low-income	Sub-Saharan Africa	6.4666667
6	Low-income	East Asia & Pacific	6.4666667
7	Low-income	Middle East & North Africa	3.4666667
8	Low-income	South Asia	4.7666667
9	Low-income	Sub-Saharan Africa	6.6666667
10	Lower-middle-income	East Asia & Pacific	5.4666667
11	Lower-middle-income	Europe & Central Asia	5.5166667
12	Lower-middle-income	Latin America & Caribbean	5.4666667
13	Lower-middle-income	Middle East & North Africa	4.4666667
14	Lower-middle-income	North America	5.4666667
15	Lower-middle-income	Sub-Saharan Africa	5.4666667
16	Not Available	East Asia & Pacific	15.7666667
17	Not Available	Europe & Central Asia	5.4666667
18	Not Available	Latin America & Caribbean	5.4666667
19	Not Available	Sub-Saharan Africa	5.4666667
20	Upper-middle-income	East Asia & Pacific	9.4666667
21	Upper-middle-income	Europe & Central Asia	11.4666667
22	Upper-middle-income	Latin America & Caribbean	9.4666667
23	Upper-middle-income	Middle East & North Africa	4.4666667
24	Upper-middle-income	South Asia	4.7666667
25	Upper-middle-income	Sub-Saharan Africa	4.4666667

Suicide Rate by Income Group

```
# plotting suicide data by income group.
sns.boxplot(x='Income Group', y='Suicide', data=mean_ca)
```

```
    /usr/local/lib/python3.7/dist-packages/seaborn/
```



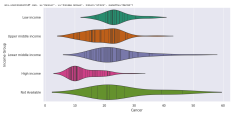

```
df_can = df6[['Income Group', 'Cancer']]
df_can
```

	Income Group	Cancer
0	Low-income	10.7
1	Upper-middle-income	17.4
2	Lower-middle-income	13.6
3	High-income	19.6
4	Lower-middle-income	14.2
...	...	...
186	Upper-middle-income	13.6
188	Lower-middle-income	17.2
189	Low-income	17.5
190	Lower-middle-income	14.9
191	Lower-middle-income	14.4

188 rows × 2 columns

Income Group vs Cancer Deaths

```
# plotting Income Group vs Cancer deaths.
from seaborn.categorical import _CategoricalPlotter, catplot
import seaborn as sns
figsize = (10, 5.5 * len(df_can['Income Group'].unique()))
g = sns.catplot(data=df_can, kind="violin",
x="Cancer", y="Income Group", hue=None,
order=['Low-income', 'Upper-middle-income',
'Lower-middle-income', 'High-income', 'Not Available'],
    /opt/hostedtoolcache/Python/3.7.12/x64/lib/pkg
    Passing (positio without assigning \'hue\' is deprecated and will be removed in v0.14.0. Assign the \'y\' variable to \'hue\' and set \'legend=False\' for the same effect.
      sns.violinplot(df_can, x="Cancer", y="Income Group", order=['Low-income',
```




```
mean_ca = df6.groupby('Region')['Cancer'].mean()
mean_ca
```

```
Region
East Asia & Pacific                 20.400000
Europe & Central Asia               16.400000
Latin America & Caribbean           16.400000
Middle East & North Africa          20.400000
North America                       20.400000
South Asia                          16.400000
Sub-Saharan Africa                  20.400000
Name: Cancer, dtype: float64
```

```
df9 = df6[['Region', 'Income Group', 'Homicide', 'Child mortality']]
df9
```

	Region	Income Group	Homicide	Child mortality
0	South Asia	Low-income	3.1	109.4
1	Europe & Central Asia	Upper-middle-income	0.9	17.6
2	Middle East & North Africa	Lower-middle-income	2.2	16.4
3	Europe & Central Asia	High-income	0.65	3.9
4	Sub-Saharan Africa	Lower-middle-income	17.2	107.4
...	...	...	...	...
186	Latin America & Caribbean	Upper-middle-income	14.5	17.7
188	East Asia & Pacific	Lower-middle-income	0.1	17.5
189	Middle East & North Africa	Low-income	10.7	18.2
190	Sub-Saharan Africa	Lower-middle-income	13.7	107.5
191	Sub-Saharan Africa	Lower-middle-income	17.2	108.3

188 rows × 4 columns

```
mean_c = df9.groupby('Region')['Homicide'].mean()
mean_c
```

```
Region
East Asia & Pacific                 21.100000
Europe & Central Asia               21.100000
Latin America & Caribbean           21.100000
Middle East & North Africa          21.100000
North America                       21.100000
South Asia                          21.100000
Sub-Saharan Africa                  20.966667
Name: Homicide, dtype: float64
```

```
mean_c = df9.groupby('Region')['Child mortality'].mean()
mean_c
```

```
Region
East Asia & Pacific                 101.133333
Europe & Central Asia               101.133333
Latin America & Caribbean           101.133333
Middle East & North Africa          101.133333
North America                       101.133333
South Asia                          101.133333
Sub-Saharan Africa                  101.133333
Name: Child mortality, dtype: float64
```

df10.shape()

	Healthcare and access	Region	Income Group	Life Expectancy	Healthy Life Expectancy	Cancer	Suicide	Alcohol Consumption	Road Accidents	Obesity	Malaria	Hypertension	Tuberculosis	Hepatitis
0	Afghanistan	South Asia	Low-income	63.0	50.9	10.7	4.7	0.1	17.9	5.9	0.1	20.0	108.0	10.0
1	Albania	Europe & Central Asia	Upper-middle-income	76.5	66.1	17.4	14.5	9.7	11.7	47.7	0.01	27.0	7.0	2.0
2	Algeria	Middle East & North Africa	Lower-middle-income	76.7	67.1	13.6	2.3	2.6	19.9	27.1	0.1	30.0	34.0	3.0
3	Andorra	Europe & Central Asia	High-income	83.6	74.6	19.6	9.05	11.7	5.6	34.6	0.1	27.0	0.9	0.4
4	Angola	Sub-Saharan Africa	Lower-middle-income	61.1	52.8	14.2	6.7	6.8	24.7	8.2	18.0	34.7	164.0	18.0

```
df9 = df9[['Region', 'Income Group', 'Road Accidents']]
df9
```

	Region	Income Group	Road Accidents
0	South Asia	Low-income	17.9
1	Europe & Central Asia	Upper-middle-income	11.7
2	Middle East & North Africa	Lower-middle-income	19.9
3	Europe & Central Asia	High-income	5.6
4	Sub-Saharan Africa	Lower-middle-income	24.7
...	...	...	...
186	Latin America & Caribbean	Upper-middle-income	19.5
188	East Asia & Pacific	Lower-middle-income	19.5
189	Middle East & North Africa	Low-income	19.5
190	Sub-Saharan Africa	Lower-middle-income	19.5
191	Sub-Saharan Africa	Lower-middle-income	17.9

188 rows × 3 columns

```
mean_r = df9.groupby('Region')['Road Accidents'].mean()
mean_r
```

```
Region
East Asia & Pacific                 19.100000
Europe & Central Asia               19.100000
Latin America & Caribbean           19.100000
Middle East & North Africa          19.100000
North America                       19.100000
South Asia                          19.100000
Sub-Saharan Africa                  19.100000
Name: Road Accidents, dtype: float64
```

```
mean_r = df9.groupby('Income Group')['Road Accidents'].mean()
mean_r
```

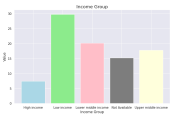
```
Income Group
High-income         7.133333
Low-income         19.133333
Lower-middle-income 19.133333
Not Available       19.133333
Upper-middle-income 19.133333
Name: Road Accidents, dtype: float64
```

```
import seaborn.pyplot as plt

# Income Group data
income_group = ['High Income', 'Low Income', 'Lower middle Income', 'Not Available', 'Upper middle Income']
values = [7.5000000, 29.000000, 20.000000, 15.000000, 18.000000]

# Create a bar chart
plt.figure(figsize=(8,6))
plt.bar(income_group, values, color=['lightblue', 'lightgreen', 'lime', 'gray', 'lightyellow'])

# Add title and labels
plt.title('Income Group', fontsize=20)
plt.xlabel('Income Group', fontsize=14)
plt.ylabel('Value', fontsize=14)
plt.show()
# Show the chart
plt.show()
```




```
df[cat = df['Region', 'Income Group', 'Sanitation']]
df.head
```

	Region	Income Group	Sanitation
0	North Asia	Low Income	19.0
1	Australia & New Zealand	Upper middle Income	3.0
2	Central Asia & Caucasus	Upper middle Income	5.0
3	Europe & Central Asia	High Income	100.0
4	Sub-Saharan Africa	Lower middle Income	20.0
...	...	...	...
195	Latin America & Caribbean	Upper middle Income	5.0
196	East Asia & Pacific	Lower middle Income	5.0
197	Middle East & North Africa	Low Income	70.0
198	Sub-Saharan Africa	Lower middle Income	20.0
199	Sub-Saharan Africa	Lower middle Income	20.0

[200 rows x 3 columns]

```
mean_sn = df[['Region', 'Income Group']]['Sanitation'].mean()
mean_sn
```

```
Income Group
Low Income                 9.250000
Lower middle Income       20.125000
Not Available             15.000000
Upper middle Income       18.000000
High Income               7.500000
Name: Sanitation, dtype: float64
```

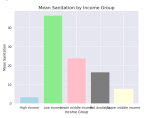
```
import seaborn.pyplot as plt

# Mean sanitation by income group
mean_sn = df[['Region', 'Income Group']]['Sanitation'].mean()

# Create a bar chart
plt.figure(figsize=(8,6))
plt.bar(mean_sn.index, mean_sn.values, color=['lightblue', 'lightgreen', 'lime', 'gray', 'lightyellow'])

# Add title and labels
plt.title('Mean Sanitation by Income Group', fontsize=20)
plt.xlabel('Income Group', fontsize=14)
plt.ylabel('Mean Sanitation', fontsize=14)

# Show the chart
plt.show()
```




```
mean_sn = df[['Region']]['Sanitation'].mean()
mean_sn
```

```
Region
Europe & Central Asia                 14.214000
Europe & Central Asia                 7.142500
Sub-Saharan Africa                    7.142500
Middle East & North Africa            7.142500
Europe & Central Asia                 7.142500
South Asia                            20.000000
Sub-Saharan Africa                    20.000000
Sub-Saharan Africa                    20.000000
Name: Sanitation, dtype: float64
```

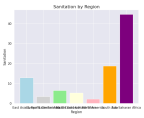
```
import seaborn.pyplot as plt

# Sanitation by region
sanitation_by_region = df[['Region']]['Sanitation'].mean()

# Create a bar chart
plt.figure(figsize=(8,6))
plt.bar(sanitation_by_region.index, sanitation_by_region.values, color=['lightblue', 'lightgreen', 'lightgray', 'lightyellow', 'lightpink', 'orange', 'purple'])

# Add title and labels
plt.title('Sanitation by Region', fontsize=20)
plt.xlabel('Region', fontsize=14)
plt.ylabel('Sanitation', fontsize=14)

# Show the chart
plt.show()
```



Regression analysis

```
X = df[['Sanitation', 'Alcohol Consumption', 'Road Accidents', 'Obesity',
y = df['Life Expectancy']
y = df['Life Expectancy']
X = df[['Sanitation', 'Alcohol Consumption', 'Road Accidents', 'Obesity',
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
# Fit the model
regressor = LinearRegression()
# Train the model
regressor.fit(X_train, y_train)
# Evaluate the model
y_pred = regressor.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
print("Mean Absolute Error:", mae)
print("Mean Squared Error:", mse)

# Print regression coefficients
print("Regression Coefficients:", regressor.coef_)
print("Regression Intercept:", regressor.intercept_)
```

```
Mean Absolute Error: 2.5000000000000004
Mean Squared Error: 6.5000000000000004
Regression Coefficients: [-0.00000000 -0.00000000 -0.00000000 -0.00000000 -0.00000000
 -0.00000000 -0.00000000]
Regression Intercept: 80.00000000000000
```

```
# Calculate the coefficients
coefficients = regressor.coef_
# Map the coefficients to the features
coefficients_dict = dict(zip(X.columns, coefficients))
# Print the coefficients
print("Coefficients:")
for feature, coefficient in coefficients_dict.items():
    print(f"{feature}: {coefficient}")
```

```
Coefficients:
Sanitation: -0.0
Alcohol Consumption: -0.0
Road Accidents: -0.0
Obesity: -0.0
Measles: -0.0
Tuberculosis: -0.0
Sanitation: -0.0
Alcohol Consumption: -0.0
```

```
# Create a heatmap of the correlation matrix
corr = df[['Sanitation', 'Alcohol Consumption', 'Road Accidents', 'Obesity',
plt.figure(figsize=(10,8))
sns.heatmap(corr, annot=True, cmap='coolwarm', cbar=True)
plt.title('Correlation Heatmap with Data Encoded')
plt.show()
```

```
[0]:
                Sanitation  Alcohol Consumption  Road Accidents  Obesity  Measles  \
Sanitation            1.00                 0.20            0.09    -0.01     0.08
Alcohol Consumption   0.20                 1.00           -0.15     0.14    -0.16
Road Accidents        0.09                -0.15            1.00    -0.12     0.07
Obesity              -0.01                 0.14           -0.12     1.00    -0.15
Measles               0.08                -0.16            0.07    -0.15     1.00
Tuberculosis          0.05                 0.08            0.01     0.07    -0.02
Sanitation            0.02                 0.02           -0.09     0.20     0.04
Alcohol Consumption   0.20                 1.00           -0.15     0.14    -0.16

                Tuberculosis  Sanitation
Sanitation              0.05        0.02
Alcohol Consumption     0.08        0.02
Road Accidents          0.01       -0.09
Obesity                 0.07        0.20
Measles                -0.02        0.04
Tuberculosis            1.00        0.05
Sanitation              0.05        1.00
Alcohol Consumption     0.08        0.02

[50 rows x 8 columns]
```

```
corr = df.corr()
corr['Sanitation'] = 0.2\nin-depth\nbetween
corr['Sanitation'].sort_values(ascending=False)
corr['Sanitation'] = 0.2\nin-depth\nbetween
corr = corr.sort_values(ascending=False)
corr
```

```
Sanitation                 NaN
Cancer                   0.020
Measles                  0.040
Sanitation               0.020
Obesity                  0.070
Measles                  0.040
Tuberculosis             0.050
Sanitation               0.020
Alcohol Consumption      0.020
```

```
import statsmodels.api as sm
# Add a constant to the model
X_const = sm.add_constant(X_train)
# Create a OLS (Ordinary Least Squares)
ols = sm.OLS(y_train, X_const).fit()
```

```
# Check the parameters obtained
ols.params
```

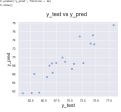
```
const                 80.000000
Sanitation            -0.000000
Cancer                -0.000000
Alcohol Consumption   -0.000000
Road Accidents        -0.000000
Obesity               -0.000000
Measles               -0.000000
Tuberculosis          -0.000000
Sanitation            -0.000000
dtype: float64
```

```
# Print a summary of the linear regression model obtained
print(ols.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:   Life Expectancy   R-squared:                       0.000
Model:                        OLS   Adj. R-squared:                  0.000
Method:             Least Squares   F-statistic:                     0.000
Date:            Mon, 04 Mar 2024   Prob (F-statistic):              1.00
Time:                    10:14:38   Log-Likelihood:                -100.0
No. Observations:              40   AIC:                             214.0
Df Residuals:                  32   BIC:                             227.1
Df Model:                       7
Covariance Type:        nonrobust
==========================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------
const         80.0000      1.000     79.919      0.000     77.959     82.041
Sanitation    -0.0000      0.000     -0.000      1.000     -0.000      0.000
Cancer        -0.0000      0.000     -0.000      1.000     -0.000      0.000
Alcohol Consumption -0.0000      0.000     -0.000      1.000     -0.000      0.000
Road Accidents -0.0000      0.000     -0.000      1.000     -0.000      0.000
Obesity       -0.0000      0.000     -0.000      1.000     -0.000      0.000
Measles       -0.0000      0.000     -0.000      1.000     -0.000      0.000
Tuberculosis  -0.0000      0.000     -0.000      1.000     -0.000      0.000
Sanitation    -0.0000      0.000     -0.000      1.000     -0.000      0.000
==============================================================================
Omnibus:                        0.000   Durbin-Watson:                   2.000
Prob(Omnibus):                  1.000   Jarque-Bera (JB):                0.000
Skew:                           0.000   Prob(JB):                        1.00
Kurtosis:                       3.000   Cond. No.                         1.00e+20
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The smallest eigenvalue is 1.41e-31. This indicates that there are
strong multicollinearity or other numerical problems.
```

```
Pkg = pd.Pkgcsv(
df['Life Expectancy'].plot.scatter(
plt.scatter(y_test, y_pred, c='blue')
plt.xlabel('y_test', fontsize = 20)
plt.ylabel('y_pred', fontsize = 20)
plt.show()
```



Checking the error terms

```
y_train_pred = regressor.predict(X_train)
res = y_train-y_train_pred
# Plot the histogram of the error terms
plt.hist(res, bin = 20)
plt.suptitle('Error Terms', fontsize = 20)
plt.xlabel('Errors', fontsize = 18)
```

```
# Classification\nplotting\nscatterplot\nmatplotlib\nseaborn\n\n\n
```